

MERGE SORT ($n \log n$)

RICORRI

1. DIVIDI A METÀ

- SE DISPARI, IL PIU' GRANDE A SINISTRA

2. RIVUNISCI IN ORDINE

QUICK SORT ($n \log n$)

1. SCEGLI L'ULTIMO COME PIVOT

2. AVVIA ITERAZIONE i, j

- i PARTE DA SINISTRA i FERMA APPENA TROVA UN MAGGIORE DEL PIVOT
- j PARTE DA DESTRA j FERMA APPENA TROVA UN MINORE DEL PIVOT

SE $i > j$

- SWAP DI i CON PIVOT

ALTRIMENTI

- SWAP DI i CON j

4. DIVIDI DALL'INIZIO A PIVOT (ESCLUSO)

E DA PIVOT (ESCLUSO) A FINE

RICORRI

EQUAZIONI ALLE RICORRENZE

1. SVOLGI

- $T(n)$, $T\left(\frac{n}{b}\right)$, $T\left(\frac{n}{b^2}\right)$

2. FORMULE IN $T(n)$

3. INDIVIDUA LA SERIE

4. ESPlicita LA SERIE IN UN POLINOMIO

- PULISCI IL POLINOMIO

5. INDIVIDUA $O()$

PER TROVARE L'ESTREMO SUPERIORE DELLA SERIE:

$$\sum_{i=0}^n x^i = \frac{x^{n+1} - 1}{x - 1}$$

$$a^{\log_b n} = n \log_b a$$

$$\log_b a = \frac{\log_k a}{\log_k b}$$

$$\sum_{i=0}^{n-1} i = \frac{1}{2} n \cdot (n-1)$$

n TERME INIZIALI

$\frac{1}{2} =$ radice iniziale
 2^i

CODICI DI HUFFMAN

1. ORDINARE GLI ELEMENTI IN BASE ALLA CHIAVE
2. UNIRE I PRIMI DUE ELEMENTI SOTTO LA SOMMA DELLE CHIAVI
 - L'ELEMENTO MINORE VA A SINISTRA
 - L'ELEMENTO MAGGIORE VA A DESTRA
3. INSERIRE L'INTERO ALBERO IN ORDINE RISPETTO ALLA RADICE
 - A SINISTRA GLI ELEMENTI MAGGIORI A SINISTRA
4. ASSEGNA 0 (SE A SINISTRA) E 1 A (DESTRA)

RIPETI

ALBERI BINARI

PRE-ORDER: INDICA LG RADICE

POST-ORDER: SOLO VERIFICA

IN ORDER: INDIVIDUA LA RADICE NEL PRE-ORDER,
A SINISTRA ABBIAMO L'ALBERO SINISTRO,
A DESTRA L'ALBERO DESTRO

BST

• INSERIMENTO IN FOGLIA:

IF (DA INSERIRE < RADICE RELATIVA)
INSERISCO A SINISTRA

IF (DA INSERIRE > RADICE RELATIVA)
INSERISCO A DESTRA

INSERISCO SOLO SE SONO IN UNA FOGLIA

• INSERIMENTO IN RADICE:

1. SE L'ALBERO E' VUOTO INSERISCO IN RADICE

1. SE NON E' VUOTO INSERISCO IN FOGLIA

2. RUOTO L'ELEMENTO FINO A PORTARLO IN RADICE

• ESPRESSIONI ALGEBRICHE:

• LE LETTERE SONO FOGLIE

• GLI OPERATORI RANCI

• SINISTRA → SINISTRA
DESTRA → DESTRA

• ESTRAZIONE SUCCESSORE:

1. TOLGO L'ELEMENTO

2. IL PRIMO PIU' GRANDE DELL'ELEMENTO DIVENTA RADICE MEDIANTE ROTAZIONI

• ESTRAZIONE PREDECESSORE:

1. TOLGO L'ELEMENTO

2. IL PRIMO PIU' PICCOLO DELL'ELEMENTO DIVENTA RADICE MEDIANTE ROTAZIONI

VISITE:

• PRE-ORDER: RADICE - SINISTRO - DESTRO | NO DO APPENA INCONTRO

• POST-ORDER: SINISTRO - DESTRO - RADICE

• IN ORDER: SINISTRO - RADICE - DESTRO

1-BST

• INSERIMENTO IN FOGLIA:

• IDENTICO AL BST, CON CHIAVE L'INTERVALLO PIU' BASSO

• MAX MEMORIZZA IL VALORE MASSIMO DI HIGH NEL SOTTOALBERO RADICATO IN UN NODE

• RICERCA:

1. VERIFICA SE IL NODE ATTUALE INTERSECA $[a, b]$

• SI $\rightarrow$ FINE

• NO



2. SE MAX DEL SOTTOALBERO SINISTRO E' $\geq a$

• SI $\rightarrow$ ESPLORA PRIMA IL SOTTOALBERO SINISTRO

• NO $\rightarrow$ VAI NEL SOTTOALBERO DESTRO

TABELLE DI HASHING

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97

NUMERI PRIMI

• N: NUMERO DI ELEMENTI

$$Q = \frac{N}{\pi} \quad \text{FATTORE DI CARICO}$$

CLUSTER: INSIEME DI NUMERI VICINI

1	2	3	4	5	6	7	8	9	10	11	12	13
A	B	C	D	E	F	G	H	I	J	K	L	M
N	O	P	Q	R	S	T	U	V	W	X	Y	Z
14	15	16	17	18	19	20	21	22	23	24	25	26

• LINEAR CHAINING:

$$h(k) = k \bmod M$$

- VETTORE DI LISTE CONTENENTI OGNUNA GLI ELEMENTI CON LA STESSA CHIAVE

- L'INSERIMENTO AVVIENE IN TESTA (NEGLIO LEGGERE AL CONTRARIO) PER INSERIRE POI IN CODA

M : PIU' PICCOLO NUMERO PRIMO $\geq \frac{N}{Q}$

LUNGHEZZA MEDIA DELLE LISTE

• LINEAR PROBING:

$$h(k) = (k + i) \bmod M$$

PIU' PICCOLO NUMERO PRIMO $\geq \frac{N}{Q}$

- EFFETTUA L'HASH CON $i=0$ ED INCREMENTO DI 1 FINO A QUANDO NON TROVO UNA CELLA LIBERA

• QUADRATIC PROBING

$$h(k) = (k + c_1 i + c_2 i^2) \bmod M$$

PIU' PICCOLO NUMERO PRIMO $\geq \frac{N}{Q}$

- EFFETTUA L'HASH CON $i=0$ ED INCREMENTO DI 1 FINO A QUANDO NON TROVO UNA CELLA LIBERA

• DOUBLE HASHING

$$\begin{cases} h(k) = (h_1(k) + i \cdot h_2(k)) \bmod n \\ h_1(k) = k \bmod n \\ h_2(k) = 1 + (k \bmod A) \end{cases}$$

$$h(k) = (k \bmod n + i (1 + (k \bmod A))) \bmod n$$

$\downarrow$
 PIÙ PICCOLO NUMERO
 PRIMO $\geq \frac{n}{2}$

$\downarrow$
 97

- EFFETTUA L'HASH CON $i=0$ ED INCREMENTO DI 1 FINO A QUANDO NON TROVO UNA CELLA LIBERA

HEAP

- LEFT = $2 \cdot i + 1$
- RIGHT = $2 \cdot i + 2$
- PARENT = $\frac{(i-1)}{2}$

MAX-HEAP $\rightarrow$ PADRE $\geq$ FIGLI

MIN-HEAP $\rightarrow$ PADRE $\leq$ FIGLI

- ALBERO COMPLETO A SINISTRA

• HEAPIFY:

IPOTESI: I FIGLI SONO GIÀ HEAP

- ↳ DAL BASSO VERSO L'ALTO
- ↳ SI APPLICA DA UNA RADICE i IN SU

1. TROVA I FIGLI DI i
2. CONFRONTA LA TERNA
3. SCAMBIA IL MAGGIORE/MINORE SE C'È
4. RIAPPLICA LA FUNZIONE SUL FIGLIO SCAMBIATO

• HEAPBUILD:

IPOTESI: LE FOGLIE SONO GIÀ HEAP VALIDI,

- ↳ DAL BASSO VERSO L'ALTO

1. SI PARTE DA $\frac{n}{2} - 1$

l'ULTIMO PADRE

2. APPLICA HEAPIFY DA I FINO ALL'ULTIMA RADICE

• INSERIMENTO IN HEAP:

1. INSERISCI IN CODA
2. CONTROLLA LA TERNA: PADRE, FRATELLO, COSA APPENA AGGIUNTA E FAI SALIRE IL MAGGIORE / MINORE
3. RIAPPLICA SULLO SCAMBIO

HEAPIFY-UP

• ESTRAZIONE IN RADICE:

1. ELIMINA LA RADICE
2. L'ULTIMO ELEMENTO VA IN RADICE
3. APPLICA HEAPIFY

• CAMBIO VALORE (PRIORITY):

1. CAMBIA IL VALORE
2. CONTROLLA LA TERNA: PADRE, FRATELLO, COSA APPENA AGGIUNTA E FAI SALIRE IL MAGGIORE / MINORE
3. RIAPPLICA SULLO SCAMBIO

HEAPIFY-UP/DOWN

• HEAP SORT:

1. SCAMBIA L'ULTIMO CON IL PRIMO ELEMENTO
• ELIMINA L'ULTIMA POSIZIONE
2. APPLICA HEAPIFY

RIPETI PER TUTTI I PASSI

PROGRAMMAZIONE DINAMICA

• PARENTESIZZAZIONE

1. SCRIVI LE MATRICI IN ORDINE

• SCRIVI LE DIMENSIONI SOTTO (COMPATIBILI)

2. • SCRIVI LA MATRICE A CON DIAGONALE 0

• SCRIVI LA MATRICE S CON (DIAGONALE+1) 1,2,3...

3. PARENTESIZZA 2 A 2

$$\begin{matrix} A_i & A_j \\ a \times b & b \times c \end{matrix}$$

$$m[i,j] = m[i,i] + m[j,j] + a \cdot b \cdot c$$

3. PARENTESIZZA 1 A 3 E 3 A 1

$$\begin{matrix} A_1(A_2 \cdot A_3) \\ a \times b & b \times c & c \times d \end{matrix}$$

$$m[1,3] = m[i,j] + \dots + a \cdot b \cdot d$$

$$\begin{matrix} (A_1 \cdot A_2) A_3 \\ a \times b & b \times c & c \times d \end{matrix}$$

$$m[1,3] = m[i,j] + \dots + a \cdot c \cdot d$$

• PRENDI IL MINIMO TRA I RISULTATI E IN $S[a,b]$ LA j DEL RISULTATO MINORE

4. CONTINUA COSÌ

5. RICOSTRUIRE LA PARENTESIZZAZIONE

• METTI LO SPLIT NELLA POSIZIONE TRA LE PARENTESI
TRA MATRICE A_i E A_j , LO SPLIT STA IN $S[i,j]$

OPPURE

$$m[i,j] = \min [m[i,k] + m[k+1,j] + p_{i-1} p_k p_j]$$

$$k \in [i, j-1]$$

• LONGEST INCREASING SUBSEQUENCE

VEITORE
LUNGHERZA
PADRE

i j

→ INIZIATO A 1

0. INIZIALIZZA IL VETTORE LUNGHERZA A 1 ED IL VETTORE PADRE A -1

1. IL PRIMO ELEMENTO HA LUNGHERZA 1, PASSO AL SUCCESSIVO

2. CONTROLLO OGNI ELEMENTO PRIMA DELLA J -ESIMO, FACENDO PARTIRE i DA 0 FINO A $J-1$

3. CERCO IL PRIMO ELEMENTO PIÙ PICCOLO DI $V[J]$

- IN CREANDO LA LUNGHERZA RISPETTO A QUEL NUMERO, CHE DIVENTA ANCHE IL PADRE

FACCIO SEMPRE
IN ORDINE DI
"RANGIARE" LA
LUNGHERZA PIÙ GRANDE

4. QUANDO $i == j$, INCREMENTO J

5. STANO, SEGUENDO PADRE L'ELEMENTO CON LUNGHERZA MASSIMA

GRAFI

• DFS:

(PRESUPPONENDO UN ORDINE DELLE CHIAVI)

1. INIZIA DAL PRIMO NODO

- ASSEGNA L'INIZIO

2. VISTO IL NODO SUCCESSIVO (SECONDO L'ORDINE)

- ASSEGNO L'INIZIO

3. CONTINUA CON TUTTI GLI ALTRI NON MAI STATI SCOPERTI

- SE NON HO ALTRO DA SCOPRIRE, ASSEGNO POST
- TORNA INDIETRO

• ORDINAMENTO TOPOLOGICO:

1. DFS

2. SCRIVO I VERTICI IN ORDINE DECRESCENTE DI POST

• TIPI DI ARCHI:

• TREE: UNISCE PADRE A FIGLIO NON SCOPERTO

• BACK: UNISCE FIGLIO A PADRE

• FORWARD: UNISCE PADRE A FIGLIO SCOPERTO

• CROSS: UNISCE COMPONENTI SCONNESSE
(TUTTI GLI ALTRI)

$$Post[V] = -1$$

$$Pre[U] > Pre[V]$$

$$Pre[U] < Pre[V]$$

• PUNTI DI ARTICOLAZIONE: NODI, CHE MANTENGO CONNESSO IL GRAFO

• NODI I QUALI FIGLI NON HANNO ARCHI DI TIPO B

• BFS:

1. PARTI DA UN VERTICE

2. DETERMINA TUTTI I VERTICI RAGGIUNGIBILI

• NON VISITARE SUBITO

3. METTANO IN UNA CODA I NODI SCOPERTI E NON VISITATI

4. ESTRAI UN NODO DALLA CODA

• COMPONENTI CONNESSE:

1. AVVIA LA DFS

• OGNI VOLTA CHE IL FOR ESTERNO CAMBIA VERTICE, NE HAI TROVATA UNA

• KOSARAJU (COMPONENTI FORTEMENTE CONNESSE):

1. GRAFO TRASPOSTO

• APPLICA DFS (CON ORDINE NORMALE)

• ORDINE TOPOLOGICO

2. GRAFO NORMALE

- APPLICA **DFS** CON IL NUOVO ORDINE TOPOLOGICO
- OGNI VOLTA CHE IL FOR ESTERNO CAMBIA VERTICE, NE HAI TROVATA UNA

MINIMUM SPANNING TREE

- CONNESSO
- ACICLICO
- $N-1$ ARCHI
- SOMMA DEI PESI MINIMO

• **KRUSKAL:**

1. ORDINA IN MANIERA CRESCENTE GLI ARCHI
2. INCLUDI IL MINORE
 - IN CASO DI PARTITA, SCEGLI A CASO
 - SENZA CREARE CICLI

- (UNION-FIND)

- ESISTONO DIVERE CONFIGURAZIONI UGUALMENTE VALIDE

• **PRIM:**

1. INIZIO DA UN DATO NODO
 - LO SEGNO VISITATO
2. SCEGLIO L'ARCO A PESO MINORE TRA QUELLI ADIACENTI A QUELLI SCOPERTI
 - SENZA CREARE CICLI
 - A PARTITA DI ORDINE (ALFABETICO)
3. SCOPRO IL VERTICE DERIVATO DALL'ARCO MINORE

CANNINI MINIMI

• **Dijkstra:**

0. INIZIALIZZO TUTTO A ∞
 - QUELLO DA CUI PARTO A 0

1. ESCRIVO IN ORDINE IL VERTICE

1. ESPLORO IN PROFONDITÀ IL VERTICE
 - SE CONVIENE, AGGIORNO IL MINIMO VALORE
 - CHIUDO IL VERTICE | DA NON TOCCARE PIÙ PIÙ
2. MI SPOSTO SUL MINORE, (A PARITÀ DI ORDINE ALFABETICO)
3. TERMINA QUANDO HO CHIUSO TUTTI I VERTICI

• BELLMAN-FORD:

1. ORDINO TUTTI GLI NODI IN ORDINE ALFABETICO
 - INIZIO DAL VERTICE SCELTO
2. ESPLORA IN AMPIEZZA ←
3. CONTROLLA IL COSTO MINIMO E AGGIORNALO
4. CONTINUA IN ORDINE ALFABETICO
5. TERMINA QUANDO RAGGIUNGO N-1 ITERAZIONI OPPURE L'ITERAZIONE RIEMANE UGUALE

QUANDO RITORNO
DA DOVE SONO
PARTITO È UN PASSO

OGNI ITERAZIONE
ESPLORO IL PRIMO LIVELLO
DI AMPIEZZA DI OGNI NODO

• DAG PESATI

1. ORDINAMENTO TOPOLOGICO
2. ESPLORO IN AMPIEZZA SECONDO L'ORDINE

CAMMINI MASSIMI

1. ORDINE TOPOLOGICO
2. SEGUENDO L'ORDINE TOPOLOGICO ESPLORO BFS ←
3. APPLICO RELAXATION INVERSA

COME DIKSTRA

